

从步骤计划到产物预览

从步骤计划到产物预览：Plan-Then-Execute 范式下的契约先行多模态升级

王敬一

2026年6月

摘要

以 Claude Code 为代表的 code agent 已能用自然语言交互，让用户无需编程即可“做出东西”。但一个根本问题始终存在：自然语言是模糊的，你说的和 Agent 理解的之间永远有 gap。当前 code agent 的验证机制——无论是事后验证、逐步确认，还是 Plan-Then-Execute 的步骤确认——要么反馈周期太长，要么验证对象不是最终产物。用户只能在 Agent 跑完全程后才能判断“这是不是我想要的”，验证成本极高。

本文在 Plan-Then-Execute 范式内提出一项关键升级：将契约从“步骤计划”升级为“多模态产物预览”，让用户在 Agent 执行前就能快速验证方向。核心贡献包括：(1) 多模态契约类型学——按任务类型自适应选择预览形式（视觉任务出低像素预览、对话 Agent 出示例对话轨迹、代码库出 API 设计+架构图）；(2) 逆向拆解执行——从预览反推步骤，确保执行不偏离已验证的方向；(3) 契约先行的适用边界分析。这一升级的核心价值在于缩短验证循环：从“执行→看结果→改描述→再执行”的慢循环，变成“出预览→看方向→改描述→再预览”的快循环。

关键词：Plan-Then-Execute、契约先行、多模态契约、逆向拆解、人机对齐、Magentic-UI

1. 引言

1.1 问题：你说的话，Agent 真的听懂了吗

以 Claude Code、Cursor 为代表的 code agent 已经将编程门槛降到了史无前例的低点——用户用自然语言描述需求，Agent 就能生成完整产物。但一个根本问题始终存在：**自然语言是模糊的，你说的和 Agent 理解的之间永远有 gap。**

你说“帮我做一个简洁的产品介绍页”，你脑子里想的是白色背景、三列布局、克制的中文字体。Agent 理解的“简洁”可能是极简主义——大量留白、无衬线字体、单色配色。两个“简洁”完全不同，但你们用的都是同一个词。

当前的 code agent 工作流中，用户只能在 Agent 跑完全程、看到最终产物后，才能发现这个 gap。然后调整描述，再跑一次，再看。这个反馈循环太慢了——每一次验证的成本是完整执行，动辄几分钟到几十分钟。用户不是在“使用”Agent，而是在“赌博”——赌 Agent 这次理解对了没有。

这不是 Agent 能力的问题，而是**验证机制的问题**。你需要一个快速、低成本的方式来回答一个简单的问题：

“这玩意儿是不是我脑子里想的那个？”

1.2 现有验证机制的局限

事后验证：Agent 跑完，你看结果。不对就改描述重跑。反馈周期 = 完整执行时间，成本太高。

逐步确认：Agent 每做一步停下来问你。这能防止方向跑偏，但频繁打断流程，且你仍然看不到最终产物——你只能判断每一步“对不对”，无法判断最终“是不是我想要的”。

Plan-Then-Execute (He et al., 2025)：Agent 先生成步骤计划，你确认后执行。确认对象是文本步骤——“先做 A，再做 B，最后做 C”。但步骤描述无法回答“最终产物长什么样”——你猜想的最终产物，和 Agent 实际做出来的，可能是两个完全不同的东西。

Karpathy 的契约先行：把计划升级为代码测试断言，机器自动验证。但断言只能验证“功能对不对”，不能验证“是不是我想要的”。一个网页所有测试都通过，但你觉得它丑——断言无法捕获“丑”。

主流工业工具的验证现状：上述问题并非学术推演，而是当前主流 code agent 的真实局限。我们考察了四个代表性工具：

- **Claude Code** (Anthropic, 2026) 提供 Plan Mode——生成文本执行计划，只读分析，人不批准不执行；正常模式下，执行后通过 `/diff` 展示代码变更。无论哪种模式，验证对象都是文本步骤和代码 diff，都不是最终产物[1]。
- **Cursor** (2026) 的 Agent/Composer 模式以 inline diff 展示代码变更，用户逐块 accept/reject。验证对象是代码 diff，且只在执行后可用。实际使用中，用户常抱怨 Agent “顺手改了你没让改的东西”——而只能在 diff 中发现，然后返工[2]。
- **Windsurf** (Codeium, 2026) 的 Cascade Agent 在执行前生成步骤计划，每步有 diff 预览和 checkpoint。其 Web Preview 功能可在 IDE 内嵌浏览器预览前端页面——这是最接近“产物预览”的机制，但仅覆盖 Web 前端，且是执行后预览，无法解决“先预览方向再执行”的问题[3]。
- **GitHub Copilot** (2026) 的 Agent 模式同样遵循“先出计划、确认后执行”的流程，验证对象为文本计划和 PR diff[4]。

四个工具的一致性在于：**验证对象始终是“怎么做”（文本步骤）或“改了什么”（代码 diff），没有一个能在执行前让用户看到“最终产物长什么样”**。当一个用户说“帮我做一个简洁的产品介绍页”，Claude Code 只能给出“步骤1：分析需求 → 步骤2：设计布局 → …”——用户无法从这些步骤中判断最终页面是否简洁。这正是本文要解决的问题。

共同问题：所有现有验证机制中，要么反馈周期太长（事后验证），要么验证对象不是最终产物（步骤确认、代码断言、代码 diff）。工业界最先进的工具和学术界最新的范式，在这一点上高度一致。用户始终缺少一个**快速、直观、以最终产物为对象的验证手段**。

1.3 本文的核心主张

本文提出：**在 Plan-Then-Execute 范式内，将契约从“步骤计划”升级为“多模态产物预览”，让用户在 Agent 执行前就能快速验证“这是不是我想要的”**。

具体来说，你描述需求后，Agent 不直接执行，而是先生成一个低成本的产物预览——网页设计出线框图，对话 Agent 出示例对话，代码库出 API 设计+架构图。你看到预览，一眼判断方向对不对。不对就调整描述，Agent 重新生成预览。几轮快速迭代后方向确认，Agent 再执行完整产物。

这个方案的核心价值在于**缩短验证循环**：从“执行→看结果→改描述→再执行”的慢循环，变成“出预览→看方向→改描述→再预览”的快循环。预览的成本远低于完整执行，所以你可以快速迭代，直到方向对了再让 Agent 放手执行。配套地，**逆向拆解**——以预览为终点，倒推出每一步需要产生什么中间状态——确保执行过程不偏离已验证的方向。第 6 节将详细展开这一策略。

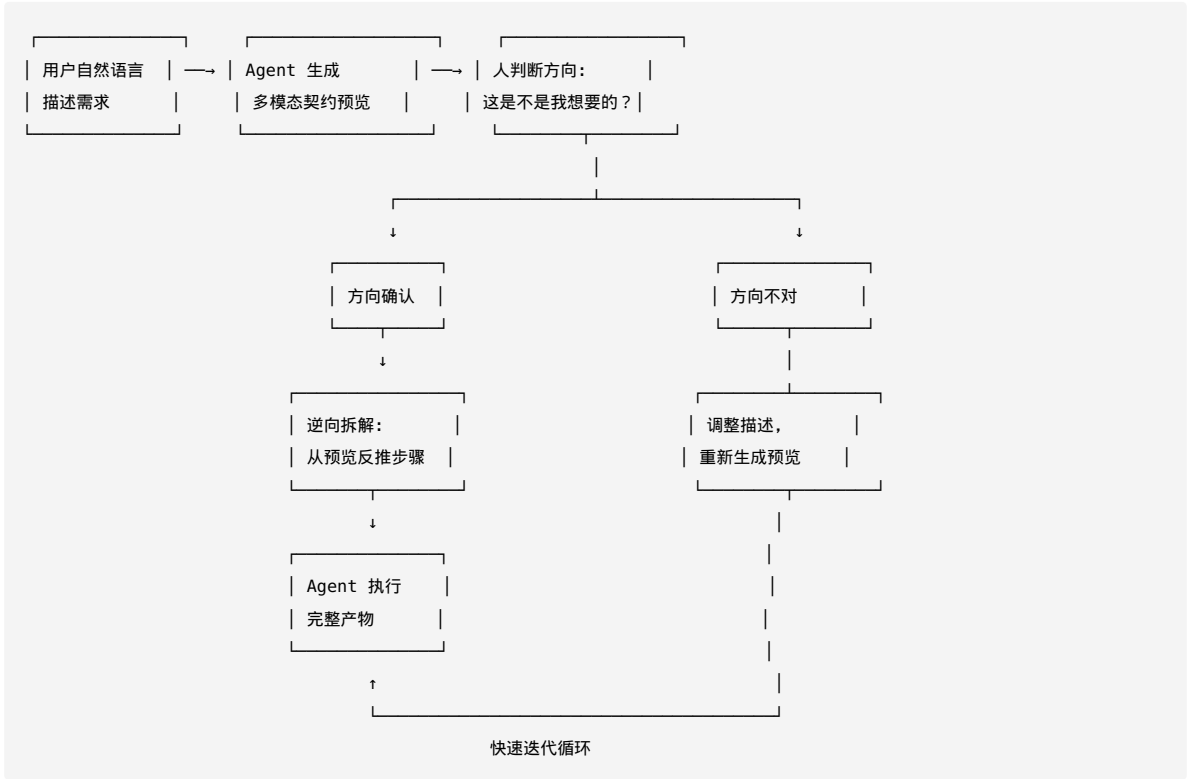


图1：契约先行多模态升级的完整流程。 核心区别在于：传统工作流中，验证发生在执行后（慢循环）；本文方案中，验证发生在执行前（快循环）。

2. 相关工作的演进

2.1 Plan-Then-Execute 范式（He et al., 2025）

He et al. (2025) 通过实证研究证实了 Plan-Then-Execute 范式的有效性：用户信任的关键节点在计划生成后、执行前。该范式引入硬门控（HARD-GATE）机制——人不确认计划，Agent 不执行。这是本文工作的范式基础。

但 Plan-Then-Execute 的“计划”是文本步骤分解。当用户看到“步骤1：分析需求 → 步骤2：设计页面布局 → 步骤3：编写 HTML 结构 → 步骤4：添加 CSS 样式 → 步骤5：测试响应式效果”，用户只能通过步骤描述猜想最终产物——不同人猜想的可能完全不同。

2.2 Karpathy 的商用级契约先行

Karpathy 在 Claude Code 的工程实践中提出了“契约先行”（contract-first）概念：在生成者写下第一行代码前，必须先与评估者达成协议——双方在磁盘上反复拉锯，直到对一组可测试的断言达成一致。Shin (2026) 将其学术化为四要素协议：written instruction contract + immutable evaluator + single editable script + complete experiment log。

Karpathy 的关键进步在于将“计划”从自然语言描述升级为**可验证的断言**——代码测试用例。这比纯文本步骤描述更精确，且可以被机器自动验证。但局限同样明显：(1) 契约形式仅限于代码测试断言，无法覆盖非代码任务；(2) 判断者是机器（测试框架）而非人，而机器能断言的东西远少于人能判断的东西；(3) 只能覆盖可测试的代码任务，对话 Agent、UI 设计、配置系统等任务完全无法适用。

2.3 Magentic-UI：最接近本文的工作（Microsoft, 2025）

Magentic-UI 是目前最接近本文思想的系统。其核心机制包括：(1) co-planning——Agent 生成文本任务分解，人编辑/批准后才执行；(2) co-tasking——任务执行中的人机协作；(3) action guards——安全护栏阻止危险操作；(4) multi-tasking——多任务并行管理。

Magentic-UI 的进步在于将 Plan-Then-Execute 中的“计划”从一次性输出升级为可交互编辑的文本任务分解，人不仅能确认，还能修改。但 Magentic-UI 的契约仍然是**文本任务分解**——“怎么做”的步骤描述，而非“做成什么样”的产物预览。一个网页好不好看、一个对话逻辑对不对、一个报告结构是否合理——用户无法从文本任务分解中直接判断。

这是本文与 Magentic-UI 的核心差异：Magentic-UI 优化了“计划的交互性”，本文优化了“计划的模态”。前者让人更容易编辑计划，后者让人能看到最终产物方向。更深层的差异在于——Magentic-UI 的文本任务分解仍然无法回答“最终产物长什么样”，而本文的多模态预览直接回答了这个问题。

2.4 本文的学术定位

本文不是提出新的范式，而是在 Plan-Then-Execute 范式内，将契约从“步骤计划”升级为“多模态产物预览”，并引入逆向拆解作为配套执行策略。下表总结了四个关键节点在学术谱系中的位置：

维度	Plan-Then-Execute	Karpathy 契约先行	Magentic-UI	本文
契约形式	文本步骤	代码断言	文本任务分解	多模态产物预览
判断者	人	机器	人	人
判断方式	步骤推理	自动化测试	步骤推理	产物预览
执行策略	正向	正向	正向	逆向拆解

3. 契约先行的核心定义与原则

3.1 形式化定义

本文将“契约先行”（Contract-First）定义为：

在 Agent 执行任务前，用最小 token 成本生成一个人可直观判断方向的产物预览，人确认方向后，从终点逆向拆解执行。

这个定义包含三个关键要素：

第一，契约是产物预览而非步骤描述。传统 Plan-Then-Execute 中的“计划”是“怎么做”（how-to-do），本文的“契约”是“做成什么样”（what-it-looks-like）。两者的区别不是语义上的，而是认知上的：看到步骤描述，人无法判断最终产物长什么样；看到产物预览，人能判断最终方向对不对。

第二，契约追求最小 token 成本。 契约不是完整产物，而是足以让人判断“方向对不对”的最小化表达。对于一个网页设计任务，契约不需要是像素级的高保真原型，而是一个线框图——它展示了布局结构、信息层级、视觉重心，但不需要颜色、字体、图片等细节。对于一个对话 Agent，契约不需要是完整的对话脚本，而是 3-5 轮关键对话——展示了风格、语气、逻辑走向。

第三，契约确认后，执行方式从正向推进变为逆向拆解。 这是契约从“步骤描述”升级为“产物预览”后自然产生的配套策略。如果契约只是步骤描述（如 Magentic-UI），逆向拆解没有意义——因为从步骤反推步骤是同义反复。只有契约是最终产物预览，逆向拆解才有明确的参照目标。

3.2 三个核心原则

原则一：人对齐优先于机器执行。 人不确认契约，Agent 不执行。这是 Plan-Then-Execute 硬门控机制的延续，但判断对象从“文本计划”变为了“多模态预览”。这一原则的工程含义是：Agent 的执行流程中，契约生成和人的确认构成一个不可跳过的门控节点。

原则二：最小 token 成本。 契约不是完整产物，而是方向判断所需的最小化信息。这一原则有两层工程含义：第一，契约生成本身的 token 消耗应远小于完整产物生成的 token 消耗，否则契约先行失去了成本优势；第二，契约的信息密度应足够高，让人在最短时间内做出准确判断。

原则三：逆向拆解。 从终点（契约预览）反推步骤，每一步的目标是“产生一个更接近契约状态的中间产物”。这一步的工程含义是：执行过程中，每一步都可以和契约对比——“这一步产生的东西，是不是契约里那个东西的前置条件？”如果答案是否定的，说明执行已经偏离。

4. 多模态契约类型学

4.1 契约的本质

契约的本质是**人可直观判断方向的最小化产物预览**。它只需要回答一个问题：“**这个方向对吗？**”

这个定义的核心在于“直观判断”——人能直接看到最终产物方向，看一眼就能判断。一个线框图让人一眼看出“左边放导航、右边放内容”和“上边放导航、下面放内容”的区别；一个 3 轮对话让人一眼看出“热情随和”和“专业克制”的区别。这种判断不需要专业训练，普通人即可完成。

我们认为，人能判断的东西远多于机器能断言的东西。 一个网页好不好看、一个对话逻辑对不对、一个报告结构是否合理——这些没法写成测试用例，但人看一眼就知道了。这正是 Karpathy 契约先行（代码断言）无法覆盖、但本文方案可以覆盖的领域。

4.2 六种契约类型

我们提出以下六种契约类型作为概念框架，按任务类型分类：

任务类型	契约形式	判断方式	典型示例
视觉任务（网页/UI/PPT）	低像素预览图	一眼看结构	线框图、布局草图
对话 Agent	示例对话轨迹	读 3 轮感受到风格	核心对话，展示逻辑和语气

代码库	API 设计 + 架构图	看接口和模块关系	接口签名 + 架构图
配置系统	预期行为表	看条件→结果映射	“开启 X 后，在 Y 条件下输出 Z”
数据分析	输出格式 + 结论方向	看表格结构和一句话	表格结构 + 一句话结论
视频/动画	故事板 / 低帧率预览	看关键帧序列	关键帧排列

这六种类型的共同特征是：**人能直接看到最终方向，而不需要通过步骤描述去猜想**。看一眼线框图就能判断布局对不对——这就是多模态契约相比步骤描述的核心优势。

4.3 契约类型的自适应选择

契约类型不是由人手动指定的，而是由 Agent 根据任务类型自适应选择。具体规则如下：

- 如果任务描述中包含“网页”“UI”“页面”“PPT”“海报”等视觉产出关键词，选择**低像素预览图**
- 如果任务描述中包含“对话”“客服”“助手”“Agent”等交互关键词，选择**示例对话轨迹**
- 如果任务描述中包含“代码”“API”“架构”“模块”等开发关键词，选择**API 设计 + 架构图**
- 如果任务描述中包含“配置”“开关”“参数”“规则”等系统关键词，选择**预期行为表**
- 如果任务描述中包含“数据”“分析”“报表”“统计”等分析关键词，选择**输出格式 + 结论方向**
- 如果任务描述中包含“视频”“动画”“演示”等动态关键词，选择**故事板预览**

自适应选择的关键在于：Agent 不需要理解“什么是好的契约”，只需要按任务类型匹配对应的契约形式。这个匹配逻辑是确定性的，不需要额外的 LLM 推理。

5. 契约的生成技术

5.1 视觉契约：文本→结构化视觉

文本→结构化视觉的技术路径已有多项成熟方案，本文不发明新图生成技术，而是复用现有成果：

- **WireGen** (He et al., 2023)：通过 LLM 微调，直接从文本描述生成中保真度 UI 线框图，证明了“文本→线框图”的可行性。
- **Sony 实验** (2024)：Claude 3.5 将 UX 需求文本直接转为 HTML/CSS 线框图，证明了通用 LLM 在无微调情况下也能完成文本→结构化视觉的转换。
- **文本→SVG/Mermaid**：纯文本模型输出结构化代码，渲染为矢量图，已被广泛使用于技术文档和架构设计中。

这些技术共同验证了“文本→结构化视觉”的技术可行性。视觉契约是六种类型中生成难度最高的——它需要从文本跨越到视觉，而其他五种类型均为纯文本输出。

5.2 文本类契约：LLM 原生能力

对话轨迹、API 设计、预期行为表、数据输出格式、故事板文本描述——这五种契约的共同特点是输出为纯文本。LLM 生成结构化文本是其核心能力，不需要额外的技术栈：

- **对话轨迹**：通过 prompt 约束输出 3-5 轮关键对话，展示 Agent 的语气和逻辑走向。

- **API 设计 + 架构图**：通过 prompt 约束输出接口签名和模块关系，架构图可借助 Mermaid 渲染。
- **预期行为表**：通过 prompt 约束输出“条件→结果”映射表。
- **输出格式 + 结论方向**：通过 prompt 约束输出表格结构和一句话结论方向。

这些契约的生成在技术上已完全成熟。关键约束是“最小 token 成本”原则——生成预览而非完整产物：3-5 轮对话而非完整对话脚本，接口签名而非完整 API 文档，条件映射表而非完整配置手册。

5.3 动态契约：故事板预览

视频/动画的契约形式是故事板或低帧率预览，目前技术成熟度最低。可能的路径包括关键帧文本描述 + 文本 → 图像生成，但帧间过渡和动态预览的连贯性仍需探索。本文将其纳入概念框架，具体生成技术留待后续工作。

5.4 编排创新：从“生成好看的东西”到“生成判断方向的预览”

本文的创新不在技术，在编排——将已有的生成技术嵌入 Plan-Then-Execute 流程，服务于一个新的目的：**不是“生成好看的图”，而是“生成让人判断方向的预览”**。

在契约先行框架下，生成技术被重新定位为**契约生成器**。这个重新定位带来了三个关键变化：

第一，质量标准的改变。传统生成技术追求画面质量（高保真、美观）或内容完整度，契约生成器追求信息密度（结构清晰、方向明确）。一个线框图不需要好看，但需要让人一眼看出布局逻辑；一个对话轨迹不需要覆盖所有场景，但需要让人感受到语气和风格。

第二，中间层文本的可编程性。视觉契约的核心路径是“文本→结构化描述（SVG/Mermaid/HTML）→渲染图”。这个结构化描述层是纯文本，可以被程序化验证（检查元素是否齐全）、被修改（人可以直接编辑文本调整布局）、被逆向拆解引用（“第三步需要生成这个 Mermaid 图里的 Module A 部分”）。这是纯文生图（文本→像素）做不到的。

第三，token 成本优势。生成一个线框图的 SVG 文本，估计约 200-500 tokens，而生成一个完整网页的代码估计约 2000-5000 tokens。契约先行将 token 消耗从“完整产物”降低到“方向预览”，这才使得“先预览再执行”在成本上可行。

6. 逆向拆解：从终点到起点

6.1 正向推进为什么容易跑偏

正向推进是当前 Agent 执行的主流策略：从起点开始，逐步向前推进，每一步的输出作为下一步的输入。以思想实验说明这种策略的根本缺陷——**偏差累积**：假设每步执行产生 ϵ 的微小偏差，经过 N 步后，累积偏差可能使最终结果完全偏离原始方向。更关键的是，正向推进中没有任何参照物——Agent 不知道“正确的方向”是什么，只能根据前一步的输出继续推进。

6.2 逆向拆解的优势

逆向拆解从根本上改变了执行策略：**从终点（契约预览）出发，反推每一步需要产生什么。**

终点状态（契约预览）→ 倒数第1步需要什么 → 倒数第2步需要什么 → ... → 起点

每一步都可以和契约对比：“这一步产生的东西，是不是契约里那个东西的前置条件？”如果答案是否定的，说明执行已经偏离，需要修正。

举例说明：用户要求“做一个电商产品详情页，左侧商品图，右侧价格+购买按钮，下方评价区”。契约生成一个线框图展示这个布局。逆向拆解的执行过程：

- **倒数第1步**：添加评价区 → 需要“右侧价格+购买按钮”已就位
- **倒数第2步**：添加右侧价格+购买按钮 → 需要“左侧商品图”已就位
- **倒数第3步**：添加左侧商品图 → 需要“页面框架”已就位
- **倒数第4步**：搭建页面框架 → 起点

每一步都有明确的参照目标（契约中对应的视觉区域），每一步的产出都可以和契约对比验证。即使某一步产生了偏差，下一步在对照契约时也能立即发现并修正。

6.3 逆向拆解与契约的依存关系

逆向拆解不是独立于契约先行的执行策略，而是契约升级为产物预览后自然产生的配套策略。如果契约只是步骤描述（如 Magentic-UI），逆向拆解没有意义——因为“从步骤反推步骤”是同义反复，无法提供新的参照信息。只有当契约是最终产物的预览，逆向拆解才能将“最终产物”作为参照目标，为每一步提供验证基准。

7. 契约先行的适用边界

契约先行并非万能。本节诚实面对其根本局限。

7.1 对话 Agent 的运行时质量

契约中的“示例对话轨迹”只能预览典型对话的风格和逻辑走向，无法保证所有对话场景的质量。对话 Agent 的运行时表现依赖于用户输入的不确定性——用户可能问出完全不在契约覆盖范围内的问题。契约先行可以确保 Agent 的“风格方向”正确，但无法确保“运行时质量”。

7.2 非视觉化产物

后端服务、基础设施配置、数据库 Schema 等任务，没有直观的契约形式。预期行为表（“开启 X 后，在 Y 条件下输出 Z”）可以覆盖配置类任务，但对于纯后端服务（如“搭建一个微服务框架”），契约先行能提供的方向指引有限。

7.3 需要体验才能判断的任务

交互流畅度、用户体验细节、动画节奏——这些需要“用一用才知道”的任务，静态预览无法提供充分的判断信息。故事板可以预览关键帧，但帧与帧之间的过渡体验无法预览。

7.4 核心限制

契约先行适用于“人看一眼就知道方向对不对”的任务。对于需要“用一用才知道”的任务，契约先行可以提供方向指引，但无法替代实际验证。这不是契约先行的缺陷，而是其能力边界。

8. 结论

本文在 Plan-Then-Execute 范式内，提出将契约从“步骤计划”升级为“多模态产物预览”并引入逆向拆解执行。三项核心贡献：

1. **多模态契约类型学**：6 种契约形式，按任务类型自适应选择，让确认对象从“步骤描述”变为“产物预览”，任何用户都能直观判断方向。
2. **逆向拆解执行策略**：从终点契约反推步骤，每一步对照契约验证，避免正向推进的累积偏差。
3. **契约先行的适用边界**：明确“人看一眼能判断”和“需要用了才知道”的边界，不夸大契约先行的适用范围。

这一升级的核心价值在于缩短验证循环：用户不再需要等 Agent 跑完全程才能判断方向，而是在执行前通过预览快速验证“这是不是我想要的”。这让 code agent 从“赌 Agent 理解对了”变成“确认方向对了再执行”。

一句话总结：Plan-Then-Execute 让人确认“怎么做”，本文让人确认“做成什么样”——前者看不到最终产物，后者能看到方向。

参考文献

- He, G. et al. (2025). Plan-Then-Execute: An Empirical Study of User Trust and Team Performance in Agent Workflows. arXiv:2502.01390.
- Karpathy, A. (2026). “契约先行” 工作流（微博文章）。
<https://m.weibo.cn/detail/5315267756558736>
- Shin, M. (2026). An Auditable AI Agent Loop for Empirical Economics. arXiv:2603.17381.
- Microsoft Research. (2025). Magentic-UI: An Experimental Human-Centered Web Agent. arXiv:2507.22358. <https://github.com/microsoft/magentic-ui>
- He, Z. et al. (2023). WireGen: Generating Wireframes from Text Descriptions. arXiv:2312.07755.
- Sony Interactive Entertainment. (2024). Using LLMs to Generate UX Wireframes.
<https://sonyinteractive.com/en/news/blog/using-llms-to-generate-ux-wireframes/>
- [1] Anthropic. (2026). Claude Code Documentation — Plan Mode & Permissions.
<https://code.claude.com/docs/en/common-workflows>
- [2] Cursor. (2026). Agent Mode Overview. <https://docs.cursor.com/agent/overview>
- [3] Codeium. (2026). Windsurf Cascade Documentation. <https://codeium.com/windsurf>
- [4] GitHub. (2026). Copilot CLI — Agent Mode & Autopilot. <https://docs.github.com/en/copilot>